DAY 6 | POST-READ

SQLAlchemy, Relationships and Docs: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
ORM	Classes map to tables, objects to rows. Engine, Base, models, Session, commit.
Columns	primary_key, unique, nullable, default, ForeignKey. Enforced by the database.
Queries	select(Model).where(...).order_by(...); s.scalars(stmt); s.get(Model, id).
One-to-many	ForeignKey on the many side, relationship() with back_populates on both.
Many-to-many	Association table of id pairs, relationship(secondary=table).
Transactions	A unit of work: commit on success, rollback on error. ACID.
OpenAPI	Machine-readable API contract at /openapi.json.
Swagger and ReDoc	/docs to try requests, /redoc to read. Both from the same schema.
Better docs	App title, version, description; tags, summary, responses, Field examples.

Part B: Going beyond today

1. from_attributes: ORM object to Pydantic

```
class BookOut(BaseModel):
    model_config = {"from_attributes": True}
    id: int
    title: str

BookOut.model_validate(book_row)    # works with SQLAlchemy objects
```

2. Cascades

relationship(back_populates="author", cascade="all, delete-orphan"): deleting an author now deletes their books too. Powerful, so use it deliberately.

3. Migrations with Alembic

create_all only creates missing tables; it never changes existing ones. In real projects, Alembic records every schema change as a versioned migration script you can apply or undo.

4. session.begin() context manager

```
with Session(engine) as s, s.begin():
    s.add(order)
    s.add(payment)
# commits automatically, or rolls back if an exception is raised
```

5. Why ORMs prevent SQL injection

Building SQL by joining strings with user input lets attackers inject commands. ORMs send values separately from the SQL text (parameterised queries), which closes this hole. You meet injection again on Day 7.

6. Generating clients from OpenAPI

Tools such as openapi-generator read /openapi.json and produce a client library for TypeScript, Java or Python.

Part C: Check yourself

- On which table does the foreign key go in a one-to-many relationship?
- Why does many-to-many need an extra table?
- What happens to the data if an exception occurs before commit?
- What is the difference between /docs and /openapi.json?
- Name three things that make API docs genuinely useful.

Part D: Practice ideas for tonight

- Add a Comment model to tickets (one-to-many) and list comments for one ticket.
- Write a bulk import function that rolls back if any single record is invalid.
- Add descriptions, examples and error responses to every endpoint in your Library API.

Coming up on Day 7: dependency injection, middleware, JWT, OWASP API Top 10, and FastAPI with MongoDB.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today